Recibe una cálida:

# ¡Bienvenida!

Te estábamos esperando 😁

alkemy

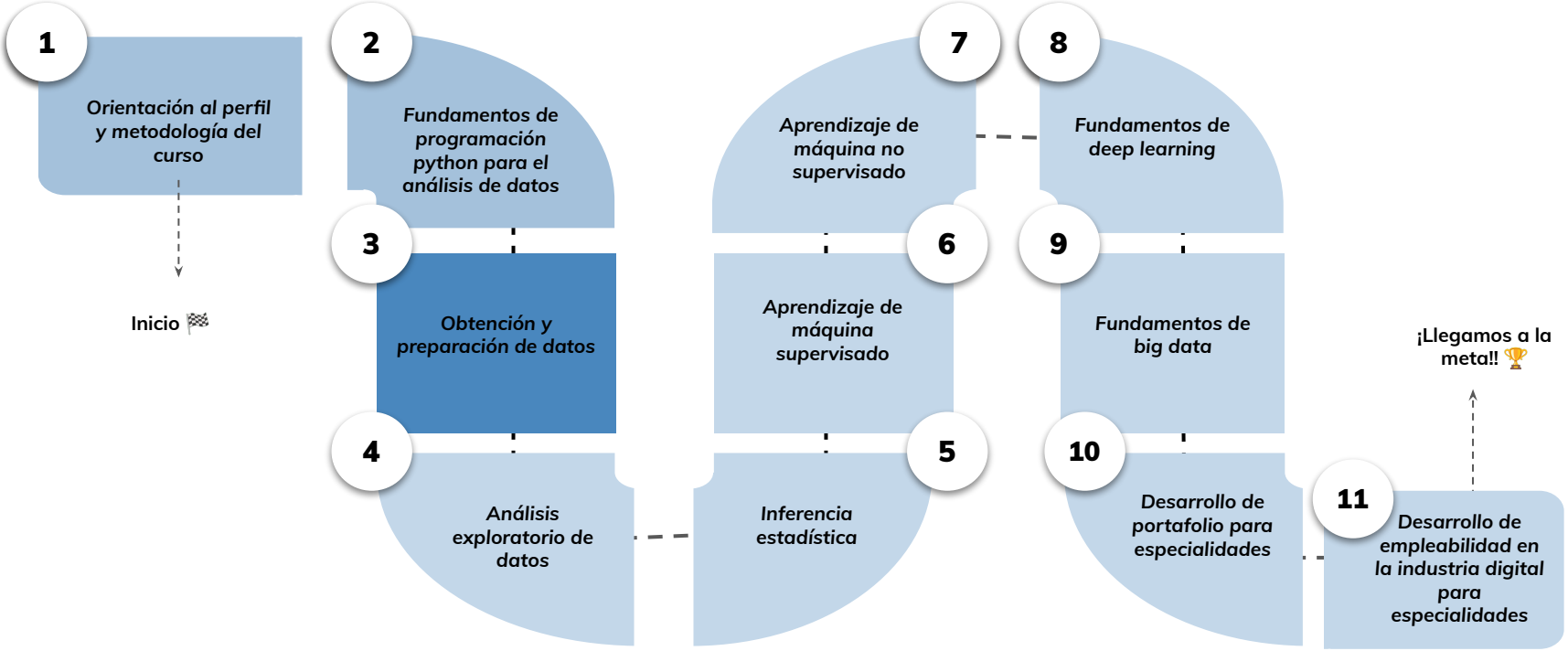
Clase grabada

# › Obtención de datos desde archivos - Parte II

**Aprendizaje Esperado 3:** Aplicar técnicas de extracción de datos desde distintas fuentes utilizando librerías utilitarias de python para su posterior uso.

# Hoja de ruta

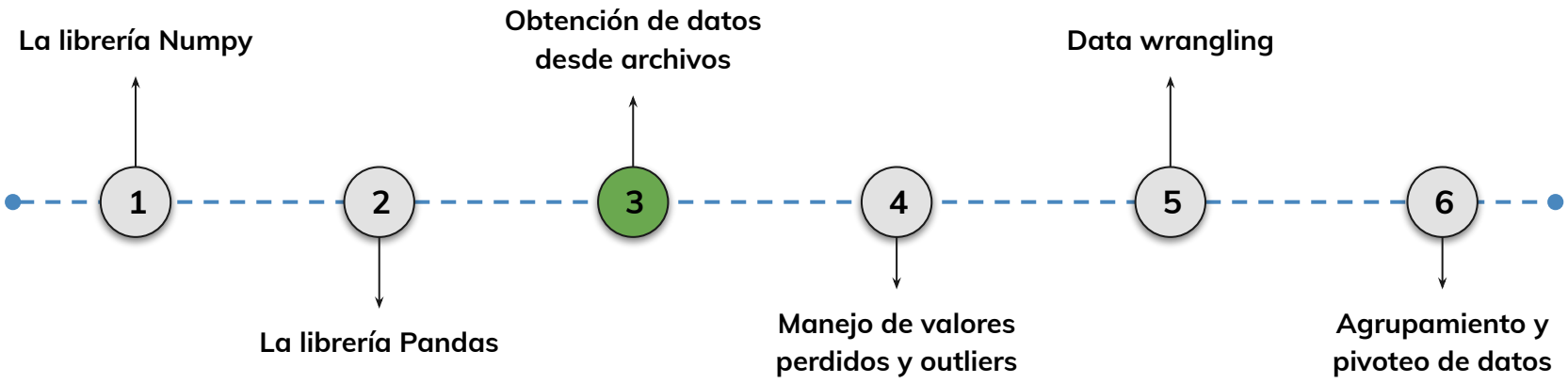
*¿Cuáles skills conforman el programa?* **Fundamentos de Ciencia de Datos**




alkemy

# Roadmap de lecciones

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*




alkemy

# Learning Path

*¿Cuáles temas trabajaremos hoy?*

## 3.

**Extracción y optimización de datos desde múltiples fuentes**

*Aprenderemos a optimizar la lectura de archivos grandes, combinar datos de distintas fuentes (CSV, Excel, Web, API y Base de Datos), manejar datos faltantes y atípicos, y automatizar estos procesos con Python.*

- **Optimización de lectura y manejo de archivos**
  - **Lectura por fragmentos y selección de columnas**
  - **Conversión de formatos y verificación de datos**
- **Automatización e integración de datos**
  - **Automatización con schedule**
  - **Uso de APIs y conexión con bases SQL**

alkemy

# Objetivos de aprendizaje

*¿Qué aprenderás?*

- Leer archivos grandes optimizando memoria y rendimiento
- Convertir entre formatos como CSV, Excel, JSON y Parquet
- Integrar datos desde múltiples fuentes (archivos, web, APIs, SQL)
- Automatizar flujos de obtención de datos con Python
- Aplicar limpieza avanzada: datos faltantes, outliers, tipado

# Repaso clase anterior

*¿Quedó alguna duda?*

En la clase anterior trabajamos 📚:

→ *Lectura y escritura de archivos CSV con pandas*

→ *Manejo de archivos Excel, incluyendo múltiples hojas y formatos*

→ *Extracción de tablas HTML desde páginas web con read_html()*

→ *Técnicas básicas de limpieza y transformación de datos con pandas*

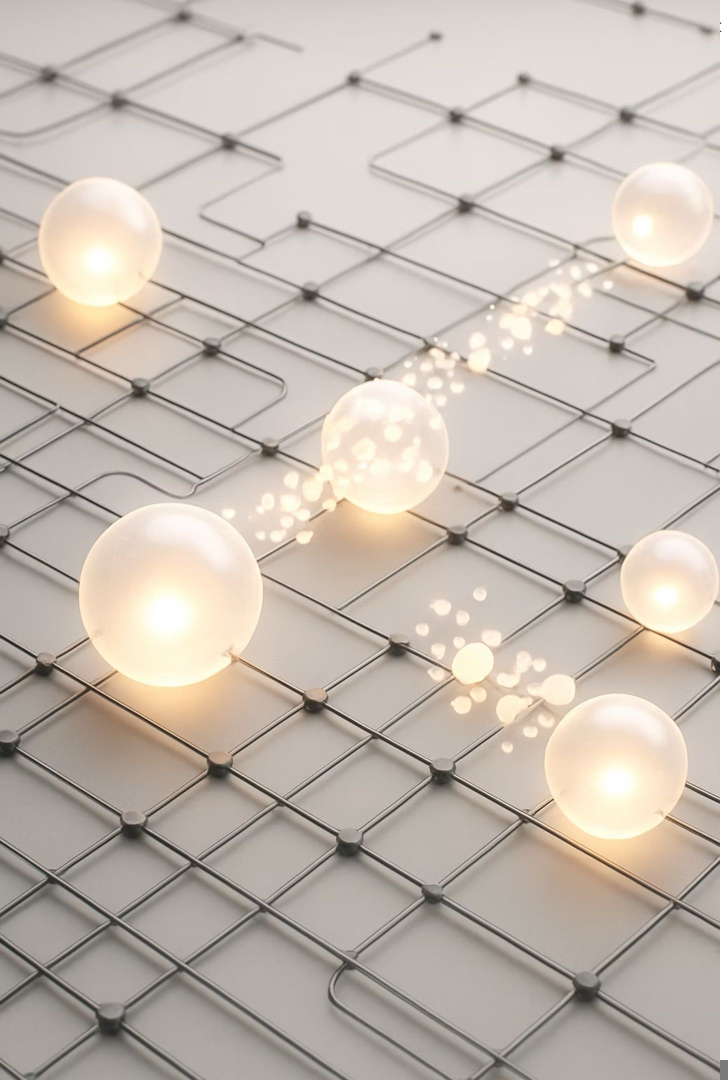
# Obtención de datos desde archivos

# Optimización de la lectura de archivos grandes

Al trabajar con archivos de gran tamaño, es importante optimizar la lectura para evitar problemas de memoria y mejorar el rendimiento.

Pandas ofrece varias opciones para manejar archivos grandes de manera eficiente:

- Lectura por fragmentos (chunks)
- Selección de columnas específicas
- Especificación de tipos de datos
- Filtrado durante la lectura



# Lectura por fragmentos (chunks)

Para archivos muy grandes, se puede leer el archivo en fragmentos más pequeños:

```
# Leer un archivo CSV en fragmentos de 1000 filas
chunks = pd.read_csv('datos_grandes.csv',
chunksize=1000)

# Procesar cada fragmento
for chunk in chunks:
 # Realizar operaciones con el fragmento
 procesado = chunk.some_operation()

 # Guardar o acumular resultados
 resultado = proceso_acumulativo(procesado)
```

alkemy

# Selección de columnas específicas

Si solo se necesitan algunas columnas del archivo, se pueden seleccionar durante la lectura para reducir el uso de memoria:

```
# Leer solo las columnas necesarias
columnas = ['Nombre', 'Edad', 'Ciudad']
df = pd.read_csv('datos.csv', usecols=columnas)
```

# Especificación de tipos de datos

Definir los tipos de datos de antemano puede mejorar significativamente el rendimiento y reducir el uso de memoria:

```
# Definir tipos de datos para cada columna
tipos = {
    'ID': 'int32',
    'Nombre': 'string',
    'Edad': 'int8',
    'Salario': 'float32',
    'Fecha': 'datetime64'
}

# Leer el archivo con los tipos especificados
df = pd.read_csv('datos.csv', dtype=tipos, parse_dates=['Fecha'])
```

ⓘ Especificar tipos de datos precisos puede reducir el uso de memoria hasta en un 50% en algunos casos.

# Filtrado durante la lectura

Es posible aplicar filtros durante la lectura para cargar solo las filas que cumplen ciertas condiciones:

```
# Función para filtrar filas durante la lectura
def filtro_edad(chunk):
    return chunk['Edad'] > 30

# Leer solo las filas que cumplen la condición
df = pd.read_csv('datos.csv',
                 chunksize=1000,
                 iterator=True)
df_filtrado = pd.concat([chunk[chunk['Edad'] > 30]
                        for chunk in df])
```

# Conversión entre formatos

### CSV

Formato simple y universal para datos tabulares

### Excel

Múltiples hojas y formato avanzado

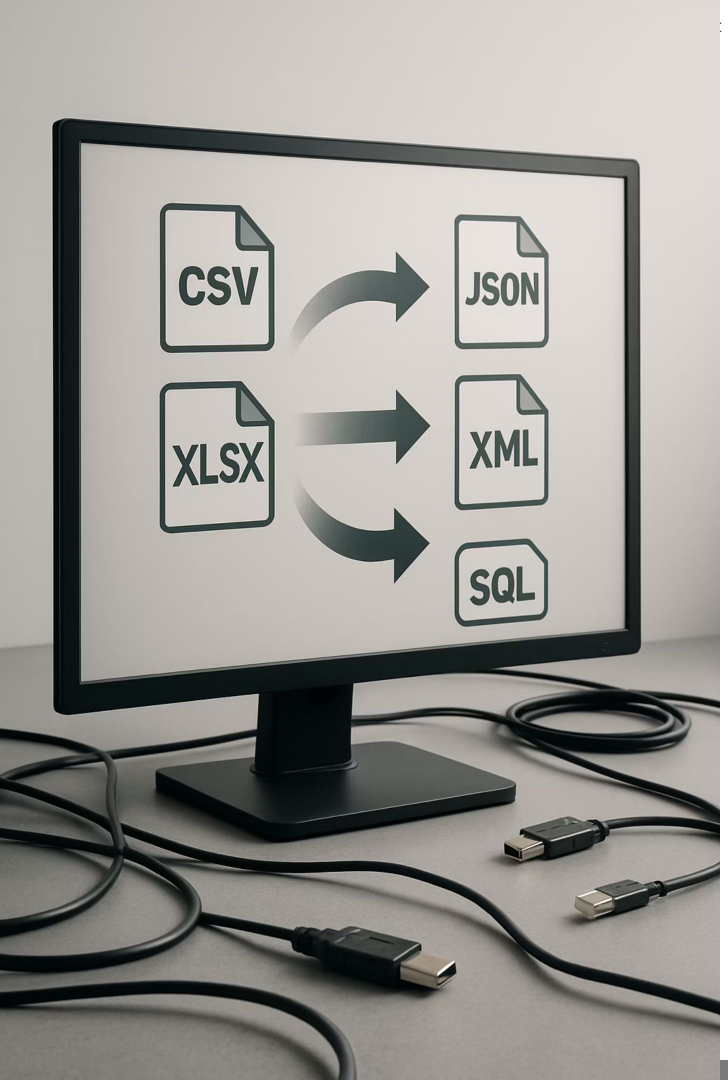
### JSON

Formato flexible para intercambio de datos

### SQL

Almacenamiento estructurado en bases de datos

Pandas facilita la conversión entre diferentes formatos de datos, permitiendo un flujo de trabajo flexible.




# Ejemplos de conversión entre formatos

```
# De CSV a Excel
df = pd.read_csv('datos.csv')
df.to_excel('datos.xlsx', index=False)

# De Excel a CSV
df = pd.read_excel('datos.xlsx')
df.to_csv('datos.csv', index=False)

# De tabla web a CSV
tablas = pd.read_html('https://example.com/tabla')
tablas[0].to_csv('datos_web.csv', index=False)

# De CSV a JSON
df = pd.read_csv('datos.csv')
df.to_json('datos.json', orient='records')
```

alkemy

# Verificación de datos cargados

## 1

### Inspección básica

Métodos para revisar rápidamente los datos

```
# Ver las primeras filas
print(df.head())

# Ver las últimas filas
print(df.tail())

# Información general del DataFrame
print(df.info())

# Estadísticas descriptivas
print(df.describe())
```

## 2

### Verificación de integridad

Comprobar problemas comunes en los datos:

```
# Verificar valores nulos
print(df.isnull().sum())

# Verificar duplicados
print(df.duplicated().sum())

# Verificar tipos de datos
print(df.dtypes)
```

# Casos de uso prácticos

### Análisis financiero

Cargar datos históricos de precios de acciones desde archivos CSV para análisis de tendencias y predicciones.

### Investigación científica

Importar resultados experimentales desde hojas de Excel para análisis estadístico y visualización.

### Análisis de mercado

Extraer tablas de precios y especificaciones de productos desde sitios web para comparativas de mercado.

### Reportes empresariales

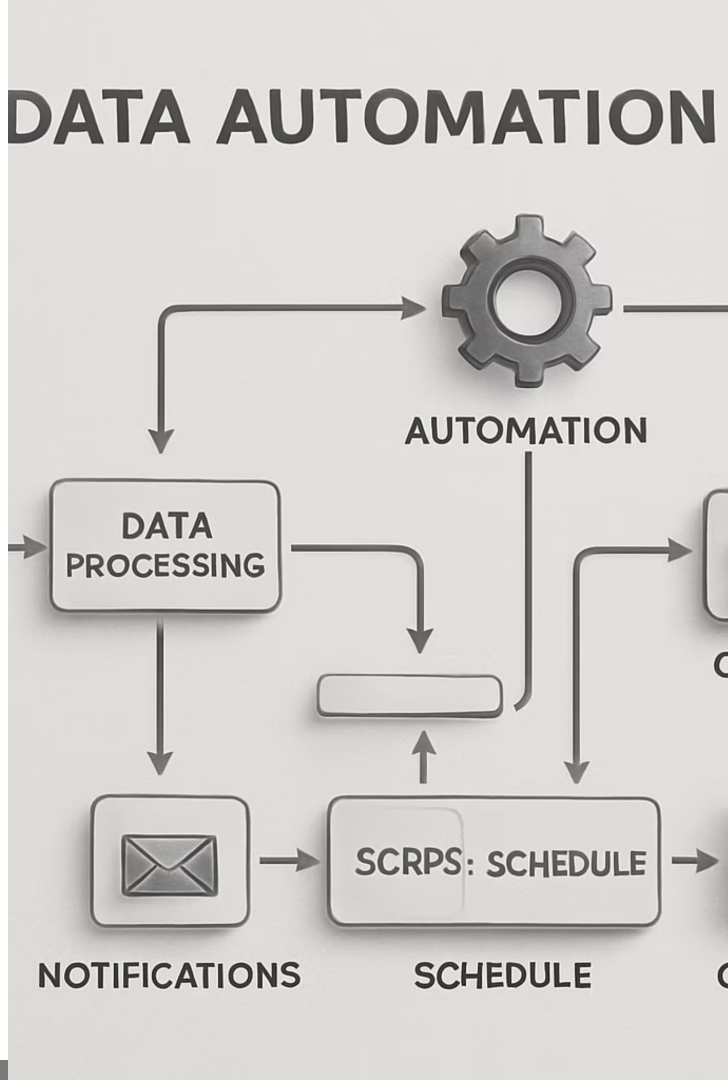
Combinar datos de ventas desde múltiples fuentes (CSV, Excel, web) para generar informes consolidados.

# Automatización de la obtención de datos

La automatización de la obtención de datos permite crear flujos de trabajo eficientes que se ejecutan periódicamente sin intervención manual.

Esto es especialmente útil para:

- Informes diarios o semanales
- Actualización de dashboards
- Monitoreo de cambios en datos
- Alimentación de modelos predictivos



```python
import pandas as pd
import schedule
import time
from datetime import datetime

def obtener_datos():
 # Obtener fecha actual
 fecha = datetime.now().strftime("%Y-%m-%d")

 # Obtener datos de diferentes fuentes
 df_ventas = pd.read_excel('ventas.xlsx')
 df_inventario = pd.read_csv('inventario.csv')
 df_precios = pd.read_html('https://ejemplo.com/precios')[0]

 # Procesar y combinar datos
 # ...

 # Guardar resultados
 resultado.to_excel(f'reporte_{fecha}.xlsx')
 print(f"Reporte generado: reporte_{fecha}.xlsx")

# Programar ejecución diaria a las 8:00 AM
schedule.every().day.at("08:00").do(obtener_datos)

# Mantener el script en ejecución
while True:
 schedule.run_pending()
 time.sleep(60)
```

# Ejemplo de script de automatización

alkemy

# Integración con otras bibliotecas

### Matplotlib/Seaborn

Visualización de datos obtenidos para análisis exploratorio y generación de gráficos.

### Scikit-learn

Aplicación de algoritmos de aprendizaje automático sobre los datos procesados.

### SQLAlchemy

Conexión con bases de datos relacionales para almacenamiento y consulta de datos.

### Dash/Streamlit

Creación de dashboards interactivos con los datos obtenidos y procesados.

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans

# 1. Obtener datos
df = pd.read_csv('clientes.csv')

# 2. Procesar datos
df = df.dropna()
df['Gasto'] = df['Gasto'].astype(float)
df['Frecuencia'] = df['Frecuencia'].astype(int)

# 3. Análisis exploratorio
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='Gasto', y='Frecuencia', hue='Categoría')
plt.title('Relación entre Gasto y Frecuencia por Categoría')
plt.savefig('analisis_exploratorio.png')

# 4. Aplicar modelo
X = df[['Gasto', 'Frecuencia']]
kmeans = KMeans(n_clusters=3, random_state=42)
df['Cluster'] = kmeans.fit_predict(X)

# 5. Guardar resultados
df.to_excel('clientes_segmentados.xlsx', index=False)
```

# Ejemplo de integración completa

# Consideraciones de seguridad

## Datos sensibles

Al trabajar con datos que contienen información personal o confidencial, es importante considerar aspectos de privacidad y cumplimiento normativo.

- Evitar almacenar contraseñas o tokens de acceso en el código
- Utilizar variables de entorno para credenciales
- Implementar técnicas de anonimización cuando sea necesario

## Web scraping

Al extraer datos de sitios web, es importante respetar los términos de servicio y las políticas de uso.

- Verificar robots.txt del sitio
- Implementar retrasos entre solicitudes
- Identificarse adecuadamente en las solicitudes

# Buenas prácticas para la obtención de datos

## 1 Documentar las fuentes de datos

Mantener un registro claro de dónde provienen los datos, incluyendo URLs, fechas de descarga y versiones de archivos.

## 2 Preservar datos originales

Guardar una copia de los datos originales sin modificar antes de realizar cualquier transformación o limpieza.

## 3 Implementar control de versiones

Utilizar sistemas como Git para rastrear cambios en los scripts de obtención y procesamiento de datos.

## 4 Automatizar pruebas de validación

Crear verificaciones automáticas para asegurar que los datos cumplen con los requisitos esperados (tipos, rangos, completitud).

# Manejo de datos faltantes

Los datos faltantes son un problema común al trabajar con archivos de datos reales. Pandas ofrece varias estrategias para detectar y manejar estos valores:

```
# Detectar valores faltantes
print(df.isnull().sum())

# Eliminar filas con valores faltantes
df_limpio = df.dropna()

# Rellenar valores faltantes
df_relleno = df.fillna(0)  # Con ceros
df_relleno = df.fillna(df.mean())  # Con la media
```

# Estrategias avanzadas para datos faltantes

## Interpolación

Útil para series temporales o datos secuenciales:

```
# Interpolación lineal
df['Columna'] =
df['Columna'].interpolate()

# Interpolación por método específico
df['Columna'] =
df['Columna'].interpolate(method='cubi
c')
```

## Imputación por KNN

Utiliza valores de registros similares:

```
from sklearn.impute import KNNImputer

imputer = KNNImputer(n_neighbors=5)
df_imputado = pd.DataFrame(
    imputer.fit_transform(df),
    columns=df.columns
)
```

## Imputación múltiple

Genera múltiples imputaciones para capturar incertidumbre:

```
from sklearn.experimental import
enable_iterative_imputer
from sklearn.impute import
IterativeImputer

imputer =
IterativeImputer(max_iter=10,
random_state=0)
df_imputado = pd.DataFrame(
 imputer.fit_transform(df),
 columns=df.columns
```

# Detección y manejo de valores atípicos

```
# Detección de outliers con el método IQR
Q1 = df['Columna'].quantile(0.25)
Q3 = df['Columna'].quantile(0.75)
IQR = Q3 - Q1

# Definir límites
limite_inferior = Q1 - 1.5 * IQR
limite_superior = Q3 + 1.5 * IQR

# Identificar outliers
outliers = df[(df['Columna'] < limite_inferior) |
 (df['Columna'] > limite_superior)]

# Eliminar o reemplazar outliers
df_sin_outliers = df[(df['Columna'] >= limite_inferior) &
 (df['Columna'] <= limite_superior)]
```

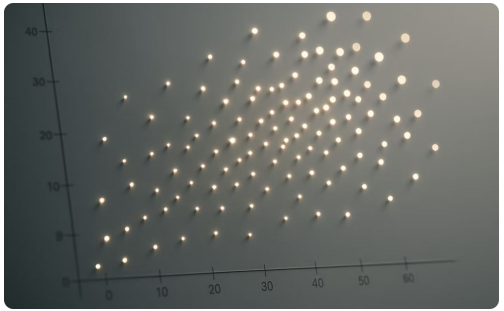
Los valores atípicos (outliers) pueden afectar significativamente el análisis de datos. Es importante detectarlos y decidir cómo manejarlos:

# Visualización de datos cargados

## Distribuciones

Histogramas y gráficos de densidad para visualizar la distribución de variables numéricas.

```
df['Columna'].hist()
```



## Correlaciones

Gráficos de dispersión y matrices de correlación para identificar relaciones entre variables.

```
sns.heatmap(df.corr(),
annot=True)
```

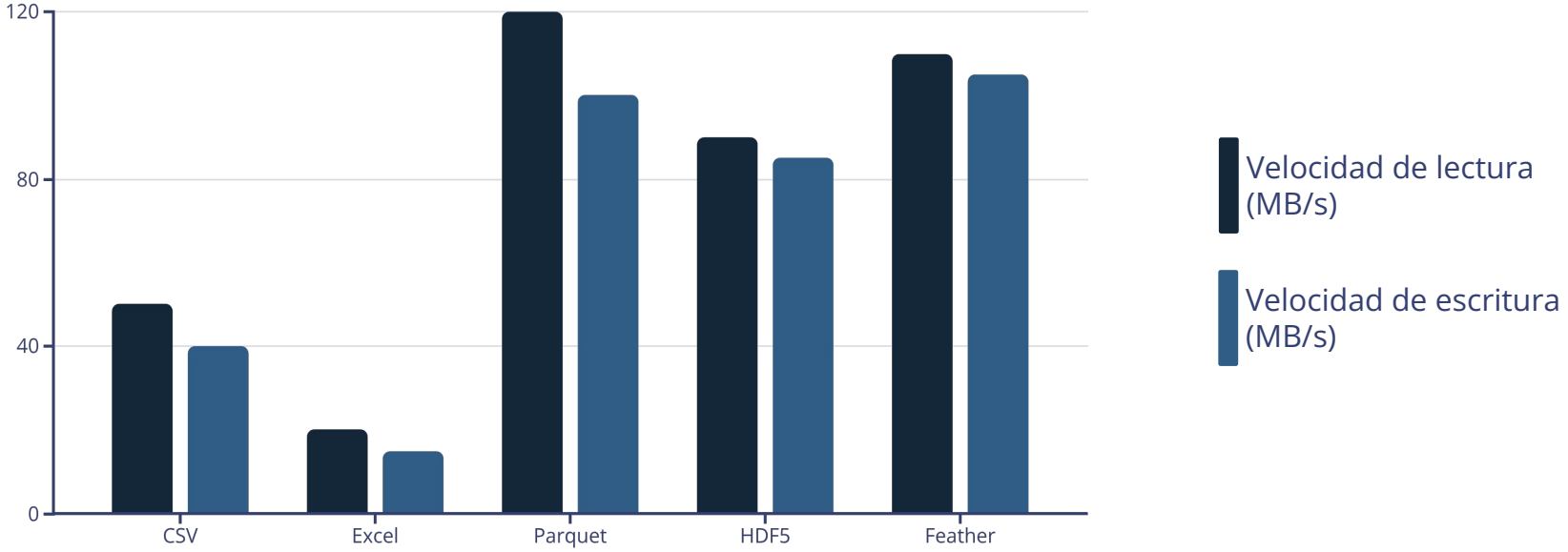
## Series temporales

Gráficos de líneas para visualizar tendencias y patrones a lo largo del tiempo.

```
df.plot(x='Fecha',
y='Valor')
```

alkemy

# Rendimiento de Pandas con diferentes formatos




El gráfico muestra el rendimiento comparativo de Pandas al leer y escribir diferentes formatos de archivo. Los formatos binarios como Parquet y Feather ofrecen un rendimiento significativamente mejor que CSV y Excel.

# Formatos alternativos para grandes volúmenes de datos

1

## Parquet

Formato columnar optimizado para análisis, con alta compresión y rendimiento.

```
# Leer archivo Parquet
df =
pd.read_parquet('datos.parque
t')# Escribir archivo
Parquetdf.to_parquet('datos.p
arquet')
```

2

## HDF5

Formato jerárquico para almacenar grandes cantidades de datos numéricos.

```
# Leer archivo HDF5
df = pd.read_hdf('datos.h5',
key='df')
# Escribir archivo
HDF5df.to_h
df('datos.h5', key='df')
```

3

## Feather

Formato rápido para intercambio de datos entre Python y R.

```
# Leer archivo Feather
df =
pd.read_feather('datos.feathe
r')
# Escribir archivo Feather
df.to_feather('datos.feather'
)
```

alkemy

# Conexión con bases de datos

Además de archivos, Pandas puede conectarse directamente con bases de datos relacionales para leer y escribir datos:

```
import pandas as pd
import sqlite3

# Conexión a base de datos SQLite
conn = sqlite3.connect('base_datos.db')

# Leer datos desde una tabla
df = pd.read_sql('SELECT * FROM tabla', conn)

# Escribir datos a una tabla
df.to_sql('nueva_tabla', conn, if_exists='replace')

# Cerrar conexión
conn.close()
```

# Conexión con otras bases de datos

## MySQL

```
import pymysql
conn = pymysql.connect(
    host='localhost',
    user='usuario',
    password='contraseña',
    database='base_datos'
)
df = pd.read_sql('SELECT *
FROM tabla', conn)
```

## PostgreSQL

```
import psycopg2
conn = psycopg2.connect(
    host='localhost',
    user='usuario',
    password='contraseña',
    database='base_datos'
)
df = pd.read_sql('SELECT *
FROM tabla', conn)
```

## SQL Server

```
import pyodbc
conn = pyodbc.connect(
 'DRIVER={SQL Server};'
 'SERVER=servidor;'
 'DATABASE=base_datos;'
 'UID=usuario;'
 'PWD=contraseña'
)
df = pd.read_sql('SELECT *
FROM tabla', conn)
```

# Obtención de datos desde APIs

Muchos servicios web ofrecen APIs que permiten acceder a datos de forma programática. Pandas puede trabajar con estos datos una vez obtenidos:

```
import requests
import pandas as pd

# Realizar solicitud a la API
url = 'https://api.ejemplo.com/datos'
respuesta = requests.get(url)
datos = respuesta.json()

# Convertir a DataFrame
df = pd.DataFrame(datos)

# Procesar y guardar
df.to_csv('datos_api.csv', index=False)
```

alkemy

# APIs populares para ciencia de datos

## Financieras

Yahoo Finance, Alpha Vantage, Quandl para datos de mercados financieros y acciones.

## Meteorológicas

OpenWeatherMap, Weather API para datos climáticos históricos y pronósticos.

## Redes sociales

Twitter API, Reddit API para análisis de sentimiento y tendencias.

## Gubernamentales

Data.gov, Eurostat, APIs de bancos centrales para datos económicos y demográficos.

# Ejemplo de obtención desde Yahoo Finance

```
import pandas as pd
import yfinance as yf

# Obtener datos históricos de una acción
ticker = 'AAPL'
periodo = '1y' # 1 año
df = yf.download(ticker, period=periodo)

# Mostrar los primeros registros
print(df.head())

# Guardar en CSV
df.to_csv(f'{ticker}_historico.csv')

# Calcular rendimientos diarios
df['Rendimiento'] = df['Close'].pct_change() * 100

# Visualizar rendimientos
import matplotlib.pyplot as plt
plt.figure(figsize=(12, 6))
df['Rendimiento'].plot()
plt.title(f'Rendimiento diario de {ticker}')
plt.savefig(f'{ticker}_rendimiento.png')
```

# Combinación de datos de múltiples fuentes

### Datos de ventas (CSV)

Registros históricos de ventas por producto y región

### Catálogo de productos (Excel)

Información detallada de productos, categorías y precios

### Datos de competidores (Web)

Precios y ofertas de competidores extraídos de sus sitios web

### Análisis integrado

Combinación de todas las fuentes para análisis competitivo completo

```python
import pandas as pd

# 1. Cargar datos de ventas desde CSV
df_ventas = pd.read_csv('ventas.csv')

# 2. Cargar catálogo de productos desde Excel
df_productos = pd.read_excel('productos.xlsx')

# 3. Cargar datos de competidores desde web
url = 'https://ejemplo.com/tabla-precios'
df_competidores = pd.read_html(url)[0]

# 4. Limpiar y preparar datos
df_ventas['Fecha'] = pd.to_datetime(df_ventas['Fecha'])
df_competidores.columns = ['Producto', 'Precio_Competidor']

# 5. Combinar datos mediante joins
df_combinado = df_ventas.merge(
 df_productos,
 on='ID_Producto',
 how='left'
)

df_final = df_combinado.merge(
 df_competidores,
 left_on='Nombre_Producto',
 right_on='Producto',
 how='left'
)

# 6. Guardar resultado final
df_final.to_excel('analisis_integrado.xlsx', index=False)
```

# Código para combinar múltiples fuentes

alkemy

# Tendencias futuras en obtención de datos

### Datos en la nube

Integración directa con servicios de almacenamiento en la nube como S3, Azure Blob Storage y Google Cloud Storage.

### Procesamiento en tiempo real

Análisis de flujos de datos continuos mediante integración con tecnologías como Kafka y Spark Streaming.

### Automatización inteligente

Uso de IA para detectar automáticamente formatos, limpiar datos y sugerir transformaciones óptimas.

### Datos no estructurados

Mejores herramientas para trabajar con datos no estructurados como texto, imágenes y audio en flujos de análisis.

# Resumen de métodos de obtención de datos

| Método | Función | Uso típico |
|---|---|---|
| read_csv() | Leer archivos CSV | Datos tabulares simples |
| read_excel() | Leer archivos Excel | Hojas de cálculo con múltiples pestañas |
| read_html() | Extraer tablas de páginas web | Datos publicados en sitios web |
| read_sql() | Leer datos de bases de datos SQL | Datos almacenados en sistemas relacionales |
| read_json() | Leer archivos JSON | Datos de APIs y configuraciones |
| read_parquet() | Leer archivos Parquet | Grandes volúmenes de datos analíticos |

# Live Coding

**¿En qué consistirá la Demo?**

*Vamos a construir un script que una datos de ventas (CSV), productos (Excel) y precios de la web. Procesará la información, detectará outliers, limpiará datos faltantes y generará automáticamente un informe diario en Excel.*

1. *Importar librerías necesarias (pandas, schedule, datetime)*
2. *Definir función obtener_datos() para ejecutar el proceso*
3. *Leer datos desde CSV, Excel y Web*
4. *Combinar DataFrames con merge()*
5. *Limpiar datos (tipos, nulos, duplicados)*
6. *Detectar outliers con método IQR*
7. *Generar archivo Excel con fecha actual*
8. *Programar ejecución diaria con schedule*

**Tiempo:** 20 minutos

Momento:

# Time-out!

⏰5 -10 min.

alkemy

# #Momentode Preguntas...

¿?

*¿Qué ventaja ofrece chunksize en la lectura de archivos?*

¿?

*¿Qué diferencias hay entre dropna() y fillna()?*

¿?

*¿Cómo se combinan datos de distintas fuentes con merge()?*

¿?

*¿Qué librerías usaste para visualizar o automatizar la carga de datos?*

Ejercicio N° 1

# Tu propio integrador de datos

# Tu propio integrador de datos

**Contexto:** 🙌

Una empresa necesita consolidar su información de productos, ventas y precios de competidores desde distintas fuentes de datos. Tu tarea será integrarlas y generar un informe automatizado.

**Consigna:** ✍️

- Leer un archivo CSV con registros de ventas.
- Leer un archivo Excel con el catálogo de productos.
- Extraer precios actuales desde una tabla web (Wikipedia, o sitio ficticio).
- Unificar las fuentes en un solo DataFrame.
- Guardar el resultado final como Excel.

**Tiempo** 🕐**:** *45 minutos*

**Paso a paso:** ⚙️

1. Crear entorno en Google Colab
2. Usar read_csv() y read_excel() para cargar archivos
3. Usar read_html() para tabla web
4. Limpiar los datos (nulos, tipos, columnas innecesarias)
5. Combinar usando merge()
6. Guardar con to_excel() sin índice

# ¿Alguna consulta?

alkemy

# Resumen

*¿Qué logramos en esta clase?*

- ✓ ***Lectura eficiente con chunksize, usecols y dtype***
- ✓ ***Conversión entre CSV, Excel, JSON y Parquet***
- ✓ ***Automatización con schedule y datetime***
- ✓ ***Integración de múltiples fuentes: CSV, Excel, Web, API, SQL***
- ✓ ***Limpieza avanzada de datos: outliers, faltantes, imputación***

# ¡Ponte a prueba!

Momento de ejercitación

*Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las* ***actividades disponibles en la plataforma.*** *Estas propuestas son clave para afianzar lo trabajado y* ***forman parte obligatoria del recorrido de aprendizaje.***

💪 Análisis de caso ——— 💪 Selección Múltiple

💪 Comprensión lectora

*Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.*

alkemy

# < ¡Muchas gracias! >

alkemy